CaRtGS: Computational Alignment for Real-Time Gaussian Splatting SLAM

Dapeng Feng, Zhiqiang Chen, Yizhen Yin, Shipeng Zhong, Yuhua Qi, and Hongbo Chen

***Abstract*—Simultaneous Localization and Mapping (SLAM) is pivotal in robotics, with photorealistic scene reconstruction emerging as a key challenge. To address this, we introduce Computational Alignment for Real-Time Gaussian Splatting SLAM (CaRtGS), a novel method enhancing the efficiency and quality of photorealistic scene reconstruction in real-time environments. Leveraging 3D Gaussian Splatting (3DGS), CaRtGS achieves superior rendering quality and processing speed, which is crucial for scene photorealistic reconstruction. Our approach tackles computational misalignment in Gaussian Splatting SLAM (GS-SLAM) through an adaptive strategy that enhances optimization iterations, addresses long-tail optimization, and refines densification. Experiments on Replica, TUM-RGBD, and VECtor datasets demonstrate CaRtGS's effectiveness in achieving high-fidelity rendering with fewer Gaussian primitives. This work propels SLAM towards real-time, photorealistic dense rendering, significantly advancing photorealistic scene representation.**

***Index Terms*—Mapping, Gaussian splatting SLAM, SLAM.**

I. Introduction

SIMULTANEOUS Localization and Mapping (SLAM) is a cornerstone of robotics and has been a subject of extensive research over the past few decades [1], [2], [3], [4], [5]. The rapid evolution of applications such as autonomous driving, virtual and augmented reality, and embodied intelligence has introduced new challenges that extend beyond the traditional scope of real-time tracking and mapping. Among these challenges is the need for photorealistic scene reconstruction, which necessitates precise spatial understanding coupled with high-fidelity visual representation.

In response to these challenges, recent research has explored the use of implicit volumetric scene representations, notably Neural Radiance Fields (NeRF) [6]. While promising, integrating NeRF into SLAM systems has encountered several obstacles, including high computational demands, lengthy optimization times, limited generalizability, an over-reliance on visual cues, and a susceptibility to catastrophic forgetting [7].

In a significant breakthrough, a novel explicit scene representation method utilizing 3D Gaussian Splatting (3DGS) [8] has emerged as a potent solution. This method not only rivals the rendering quality of NeRF but also excels in processing speed, offering an order-of-magnitude improvement in both rendering and optimization tasks.

The advantages of this representation make it a strong candidate for incorporation into online SLAM systems that require real-time performance. It has the potential to transform the field by enabling photorealistic dense SLAM, thereby expanding the horizons of scene understanding and representation in dynamic environments.

However, existing Gaussian Splatting SLAM (GS-SLAM) methods [9], [10], [11], [12], [13], [14], [15], [16], [17] struggle to achieve superior rendering performance under real-time constraints when dealing with a limited number of Gaussian primitives. These issues stem from the misalignment between the computational demands of the algorithm and the available processing resources, which can lead to insufficient optimization and optimization processes. Addressing these challenges is crucial for enhancing the performance and applicability of GS-SLAM in real-time environments.

In this letter, we scrutinize the computational misalignment phenomenon and propose the **C**omputational **A**lignment for **R**eal-**T**ime **G**aussian **S**platting **S**LAM (CaRtGS) to address these challenges. Our approach aims to optimize the computational efficiency of GS-SLAM, ensuring that it can meet the demands of real-time applications while achieving high rendering quality with fewer Gaussian primitives.

Our contributions are listed as follows:

- We provide an analysis of the computational misalignment phenomenon present in GS-SLAM.
- We introduce an adaptive computational alignment strategy that effectively tackles insufficient optimization, long-tail optimization, and weak-constrained densification, achieving high-fidelity rendering with fewer Gaussian primitives under real-time constraints.
- We conduct comprehensive experiments and ablation studies to demonstrate the effectiveness of our proposed method on three popular datasets with three distinct camera types.

Received 6 October 2024; accepted 19 February 2025. Date of publication 24 February 2025; date of current version 25 March 2025. This article was recommended for publication by Associate Editor Nicola Bellotto and Editor Sven Behnke upon evaluation of the reviewers' comments. *(Corresponding authors: Hongbo Chen; Yuhua Qi.)*

Dapeng Feng, Yizhen Yin, Yuhua Qi, and Hongbo Chen are with the Sun Yat-sen University, Guangzhou, China (e-mail: fengdp5@mail2.sysu.edu.cn; yinyzh5@mail2.sysu.edu.cn; qiyh8@mail.sysu.edu.cn; chenhongbo@mail.sysu.edu.cn).

Zhiqiang Chen is with the University of Hong Kong, Hong Kong SAR, China (e-mail: zhqchen@connect.hku.hk).

Shipeng Zhong is with WeRide Inc., Guangzhou, China (e-mail: shipeng.zhong@weride.ai).

For the benefit of the research community, we release the code and accompanying videos on our project website: https://dapengfeng.github.io/cartgs.

Digital Object Identifier 10.1109/LRA.2025.3544928

2377-3766 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 03:37:58 UTC from IEEE Xplore. Restrictions apply.

II. Related Works

GS-SLAM leverages the benefits of 3DGS [8] to achieve enhanced performance in terms of rendering speed and photorealism. In this section, we conduct a concise review of both 3D Gaussian Splatting and Gaussian Splatting SLAM.

A. 3D Gaussian Splatting

3DGS [8] is a cutting-edge real-time photorealistic rendering technology that employs differentiable rasterization, eschewing traditional volume rendering methods. This groundbreaking method represents the scene as explicit Gaussian primitivies and enables highly efficient rendering, achieving a remarkable 1080p resolution at 130 frames per second (FPS) on contemporary GPUs, and has substantially spurred research advancements.

In response to the burgeoning interest in 3DGS, a variety of extensions have been developed with alacrity. Accelerating the acquisition of 3DGS scene representations is a key area of focus, with various strategies being explored. One prominent research direction is the reduction of Gaussians through the refinement of densification heuristics [18], [19], [20]. Moreover, optimizing runtime performance has become a priority, with several initiatives concentrating on enhancing the differentiable rasterizer and optimizer implementations [20], [21], [22], [23].

Motivated by these advancements, our work addresses the challenge of insufficient optimization in photorealistic rendering within real-time SLAM by utilizing splat-wise backpropagation [20]. In parallel, recent methodologies have concentrated on sparse-view reconstruction and have sought to compact the scene representation. This is achieved by training a neural network to serve as a data-driven prior, which is capable of directly outputting Gaussians in a single forward pass [24], [25], [26], [27]. In contrast, our research zeroes in on real-time dense-view and per-scene visual SLAM. This targeted focus demands an incremental photorealistic rendering output that is tailored to the unique characteristics of each scene.

B. Gaussian Splatting SLAM

3DGS [8] has also quickly gained attention in the SLAM literature, owing to its rapid rendering capabilities and explicit scene representation. MonoGS [9] and SplaTAM [10] are seminal contributions to the coupled GS-SLAM algorithms, pioneering a methodology that simultaneously refines Gaussian primitives and camera pose estimates through gradient backpropagation. Gaussian-SLAM [11] introduces the concept of sub-maps to address the issue of catastrophic forgetting. Furthermore, Loop-Splat [12], which extends the work of Gaussian-SLAM [11], employs a Gaussian splat-based registration for loop closure to enhance pose estimation accuracy. However, the reliance on the intensive computations of 3DGS [8] for estimating the camera pose of each frame presents challenges for these methods in achieving real-time performance.

To overcome this, decoupled GS-SLAM methods have been proposed [13], [14], [15], [16], [17]. Splat-SLAM [13] and IG-SLAM [14] utilize pre-trained dense bundle adjustment [1] for camera pose tracking and proxy depth maps for map optimization. RTG-SLAM [15] incorporates frame-to-model ICP for tracking and renders depth by focusing on the most prominent opaque Gaussians. GS-ICP-SLAM [16] achieve remarkably high speeds (up to 107 FPS) by leveraging the shared covariances between G-ICP [2] and 3DGS [8], with scale alignment of Gaussian primitives. Photo-SLAM [17] employs ORB-SLAM3 [3] for tracking and introduces a coarse-to-fine map optimization for robust performance.

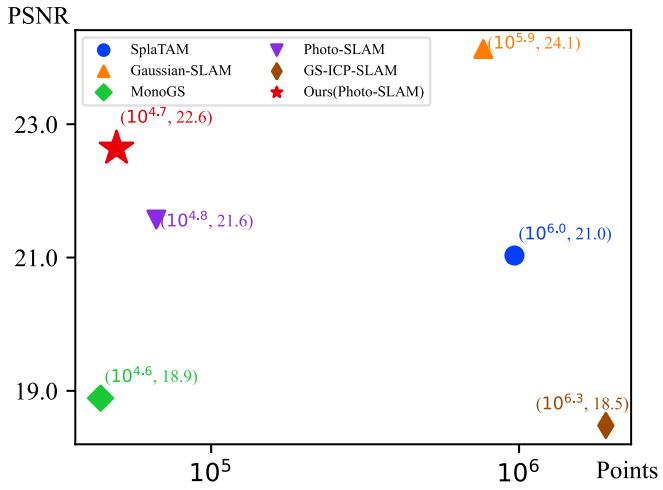


Fig. 1. **Performance on TUM-RGBD.** We provide a comparison of most of the available open-source GS-SLAM methods.

These methods achieve state-of-the-art PSNR with a large number of Gaussian primitives, as presented in Fig. 1, which will limit the application of real-time GS-SLAM in large-scale scenarios due to increased computational demands. In this letter, we delve into the limitations of existing GS-SLAM and propose an innovative computational alignment technique to enhance PSNR while reducing the number of Gaussian primitives required, all within the constraints of real-time SLAM operations.

III. Methods

In this section, we delve into the photorealistic rendering aspect of GS-SLAM. Initially, we scrutinize the computational misalignment phenomenon inherent to GS-SLAM. This misalignment can significantly impair computational efficiency and hinder the swift convergence of photorealistic rendering, adversely affecting the performance of real-time GS-SLAM. To overcome these obstacles, we propose a novel adaptive computational alignment strategy. This strategy aims to accelerate the 3DGS process, optimize computational resource allocation, and efficiently control model complexity, thereby enhancing the overall effectiveness and practicality of 3DGS in real-time SLAM applications.

A. Computational Misalignment

The computational misalignment encountered in photorealistic rendering within the context of SLAM can be attributed to three primary aspects: insufficient optimization, long-tail optimization, and weak-constrained densification, which reduces

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 03:37:58 UTC from IEEE Xplore. Restrictions apply.

rendering quality and increases map size. These factors significantly hinder the real-time applications of GS-SLAM, limiting its applicability in resource-constrained devices.

1) Insufficient Optimization: In contrast to typical 3DGS [8], which is not constrained by real-time considerations, online rendering within the realm of SLAM necessitates the concurrent execution of localization, mapping, and rendering at a speed that is synchronized with the frequency of incoming sensor data. To achieve this, the majority of current real-time GS-SLAM methods [15], [16], [17] rely on keyframes for both mapping and rendering. However, these methods typically achieve only a few thousand iterations in rendering optimization in total, which significantly lags behind the tens of thousands of iterations achieved by 3DGS [8]. Due to insufficient optimization, the optimization process has not fully converged, adversely affecting the quality of online rendering.

Recent observations by several researchers indicate that pixel-wise backpropagation in 3DGS presents a significant computational challenge [20], [21]. This process becomes a bottleneck due to the contention among multiple GPU threads for access to shared Gaussian primitives, which necessitates serialized atomic operations, thereby limiting parallelization efficiency. Unfortunately, this drawback is integrated into the previous implementations of GS-SLAM [15], [16], [17]. In this letter, we utilize a fast splat-wise backpropagation [20] to reduce thread contention. This approach not only achieves a $3\times$ increase in the number of iterations compared to the baseline [17], but also maintains the same runtime. This advancement significantly mitigates the problem of insufficient optimization, substantially improving the rendering quality of real-time GS-SLAM.

2) Long-Tail Optimization: To mitigate the issue of catastrophic forgetting, a common approach in GS-SLAM is to randomly select a keyframe from the keyframe pool for periodic reoptimization [15], [16], [17]. However, this method can result in suboptimal long-tail optimization, which overfits the oldest keyframe and underfits the newest one, as depicted in Fig. 2. Specifically, the reoptimization frequency of the earliest keyframes tends to exceed that of the most recently added ones. This disparity arises because the keyframe pool is continuously expanded as the camera moves through the environment, which can result in an uneven distribution of reoptimization efforts and and a declining trend in the PSNR for newly incoming keyframes.

In this letter, we propose an innovative adaptive optimization strategy that selects reoptimization keyframes from the pool based on their optimization loss to counteract long-tail effect. By employing this approach, we aim to increase the reoptimization frequency of keyframes with lower PSNR values. This targeted approach has been demonstrated to significantly enhance the rendering quality, as evidenced by an improvement from 34.9 dB to 36.4 dB in the Replica Room2 scenario, as depicted in Fig. 2. By doing so, our adaptive strategy ensures a more equitable distribution of reoptimization efforts across the keyframe pool, optimizing each keyframe's contribution to the system's overall performance. This innovative approach not only improves the quality of the rendered output but also enhances the efficiency and effectiveness of the reoptimization process.

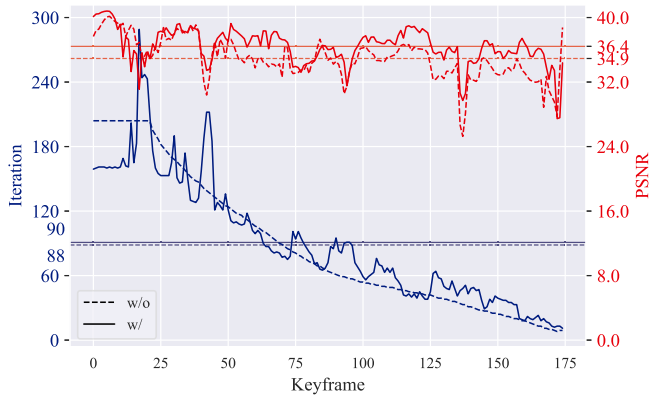


Fig. 2. **The effect of adaptive optimization on replica.** Dashed lines depict performance without adaptive optimization, while solid lines show results with it. Blue represents keyframe iterations, and red indicates PSNR. The horizontal line marks average PSNR and iterations. Our method significantly improves low-PSNR keyframe processing through enhanced iterative optimization, as evident from the trend comparison between dashed and solid lines.

3) Weak-Constrained Densification: Densification is a critical component of photorealistic rendering in the context of GS-SLAM, encompassing both geometry densification and adaptive densification [9], [10], [11], [12], [13], [14], [15], [16], [17]. Geometric densification involves the conversion of a color point cloud into initialized Gaussian primitives for each newly identified keyframe, providing a foundational geometric structure for the environment. Adaptive densification, on the other hand, refines the Gaussian primitives using operations such as splitting and cloning, which are guided by gradients and the size of the primitives themselves [8]. These densifications are solely constrained by a simplistic pruning strategy that eliminates Gaussian primitives with low opacity. However, emerging research [25], [26], [27] suggests that this approach is insufficient for managing the model's size within an optimal range. In this letter, we introduce an opacity regularization loss to encourage the Gaussian primitives to learn a low opacity, thereby not only facilitating the pruning process to eliminate less significant primitives but also preserving high-fidelity rendering.

B. System Overview

As delineated in Fig. 3, we take the modular designs, which are easy to integrate into existing real-time decoupled GS-SLAM, e.g., GS-ICP-SLAM [16] and Photo-SLAM [17].

Given a sequence of observations $\{\mathcal{V}_1, \ldots, \mathcal{V}_N\}$, we employ a state-of-the-art front-end tracker [2], [3], which estimates the 6-DoF pose for each frame and identifies keyframes $\{v_1, \ldots, v_k\}$ based on criteria related to translation and rotation. Once a keyframe v_i is identified, the frontend tracker transforms the corresponding observation $\mathcal{V}_i$ into the global coordinate system and integrates it into the global Point Cloud $\mathcal{P}$.

In the photorealistic rendering phase, we utilize 3DGS [8] as the backend render. Firstly, we convert $\mathcal{P}$ into a set of Gaussian primitives $\mathcal{G}$. Each primitive is characterized by its posistion $\mathbf{p} \in \mathbb{R}^3$, orientation represented as quaternion $\mathbf{q} \in \mathbb{R}^4$, scaling factor $\mathbf{s} \in \mathbb{R}^3$, opacity $\sigma \in \mathbb{R}^1$, and spherical harmonic coefficients

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 03:37:58 UTC from IEEE Xplore. Restrictions apply.

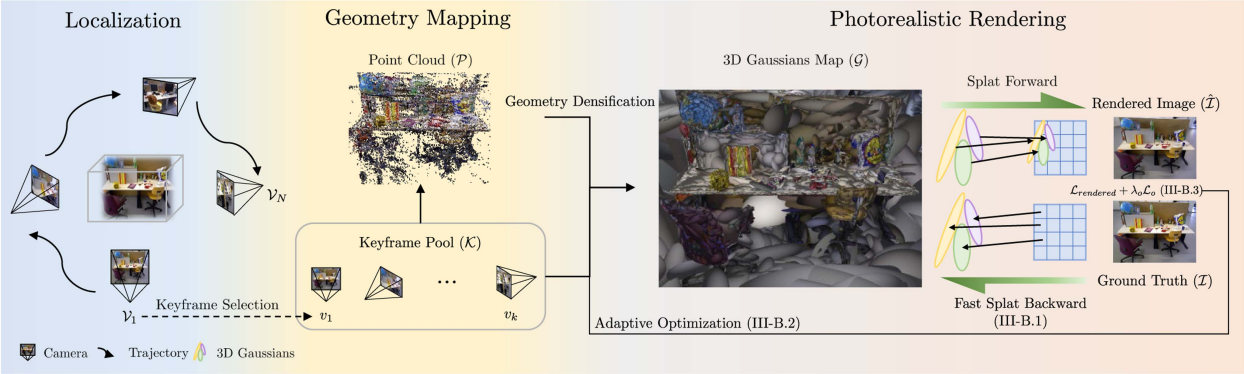


Fig. 3. **The overview of CaRtGS.** We adopt a real-time cutting-edge SLAM system as a front-end tracker, severing for localization and geometry mapping. In the photorealistic rendering back-end, we apply the proposed adaptive computational alignment strategy to enhance the 3DGS optimization process, including fast splat backward, adaptive optimization, and opacity regularization.

$\mathbf{SH} \in \mathbb{R}^{48}$. By employing α–blending rendering [8], we achieve the high-fidelity rendering $\hat{\mathcal{I}}$ for a selected keyframe v_i:

$$\hat{\mathcal{I}} = \sum_{k \in \mathcal{G}} c_k \alpha_k \prod_{j=1}^{k-1} (1 - \alpha_k), \tag{1}$$

where c_k denotes the color derived from $\mathbf{SH}$, α_k is determined by evaluating a projected 2D Gaussian multiplied with the learned opacity σ_k. To refine the Gaussian primitives $\mathcal{G}$, we take both $\mathcal{L}_1$ and Structural Similarity Index (SSIM) Loss $\mathcal{L}_{ssim}$ to supervise the optimization process. These losses are crucial for enhancing the quality of our photorealistic renderings. Additionally, we incorporate opacity regularization into our comprehensive loss function to control the model size, which is detailed in Section III-C3.

C. Adaptive Computational Alignment

To address the computational misalignment of photorealistic rendering in real-time GS-SLAM, we propose an adaptive computational alignment strategy termed CaRtGS. Below, we outline the key steps of this strategy in detail.

1) Fast Splat-Wise Backpropagation: In the conventional 3DGS optimization pipeline, the backpropagation phase is computationally demanding as it entails the propagation of gradient information from pixels to Gaussian primitives. This process necessitates the calculation of gradients for each splat-pixel pair (i, j), followed by an aggregation step. In our notation, i denotes the index of the i-th splat, and j denotes the index of the j-th pixel. To parallelize the execution, we assign thread i to process the i-th splat, and thread j to process the j-th pixel. In the forward pass, GPU thread $i+1$ applies the standard α-blending logic to transition from the received state $\mathcal{X}_{i,j}$ to $\mathcal{X}_{i+1,j}$, integrating this updated information into the gradient computation. In the backward pass, the gradients associated with the i-th splat, denoted as $\nabla \mathcal{X}_i$, are accumulated across the pixels that are influenced by this splat. This process can be mathematically represented as:

$$\mathcal{X}_{i+1,j} = \mathcal{F}(\mathcal{X}_{i,j}), \tag{2}$$

$$\nabla \mathcal{X}_{i,j} = \nabla \mathcal{F} \cdot \nabla \mathcal{X}_{i+1,j}, \tag{3}$$

$$\nabla \mathcal{X}_i = \sum_j \nabla \mathcal{X}_{i,j}, \tag{4}$$

where $\mathcal{F}$ presents the α-blending function.

Pixel-wise propagation is widely used in GS-SLAM [9], [10], [11], [12], [13], [14], [15], [16], [17], mapping threads to pixels and processing splats in reverse depth order. Thread j computes partial gradients for the splats in the order they are blended, updating the cumulative gradient for each splat through atomic operations. However, this method can lead to contention among threads for shared memory access, resulting in serialized operations that impede performance.

To address this challenge, we utilize a novel parallelization strategy [20] that shifts the focus from pixel-based to splat-based processing. This strategy allows each thread to independently maintain the state of a splat and to efficiently exchange pixel state information. Thread i can compute the gradient contribution for the i-th splat, requiring the pixel j state after the first i splats have been blended.

During the forward pass, threads archive transmittance T and accumulated color RGB for pixels every N splats, preparing for the backward pass. These stored states include initial conditions $\mathcal{X}_{0,j}, \mathcal{X}_{N,j}, \cdots \forall j$. At the commencement of the backward pass, each thread in a tile generates the pixel state $\mathcal{X}_{i,j}$. Threads then engage in rapid collaborative sharing to exchange pixel states.

For further details, please refer to Fig. 4(a). The data presented in Fig. 4(b) clearly show that the splat-wise backpropagation method significantly enhances the total number of optimization iterations by a factor of 3, increasing from an average of 4.6k to 15.4k. This improvement effectively addresses the issue of insufficient optimization compared to Photo-SLAM [17] equipped with pixel-wise propagation.

2) Adaptive Optimization: Although splat-wise propagation achieves sufficient optimization in total, the long-tail distribution of iterations per keyframe is a challenge. To address this, we recommend augmenting the splat-wise approach with an adaptive optimization based on traininig loss $\mathcal{L}$ to ensure a more equitable distribution of iterations across the keyframe pool $\mathcal{K}$.

Given a keyframe pool $\mathcal{K}_k$ containing keyframes $\{v_1, v_2, \ldots, v_k\}$, we maintain two sets: $\mathcal{R}_k = \{r_1, r_2, \ldots, r_k\}$

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 03:37:58 UTC from IEEE Xplore. Restrictions apply.

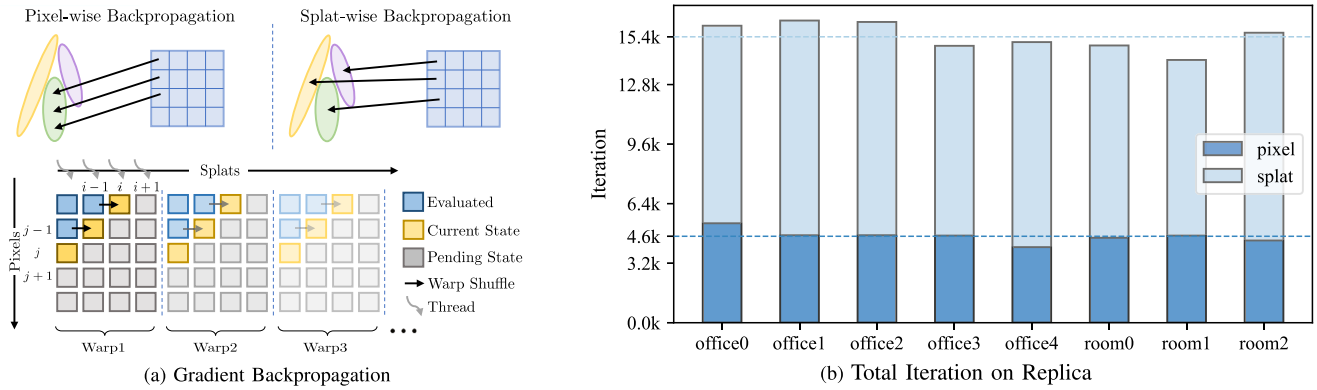


Fig. 4. **The effect of different gradient backpropagation.** (a) The original 3DGS employs pixel-wise parallelism for backpropagation, which is prone to frequent contentions, leading to slower backward passes. We introduce a splat-centric parallelism, where each thread handles one Gaussian splat at a time, significantly reducing contention. The gradient computation relies on a set of per-pixel, per-splat values, effectively traversing a splat ⇔ pixel relationship table. During the forward pass, we save pixel states for every 32nd splat. For the backward pass, splats are grouped into buckets of 32, each processed by a CUDA warp. Warps utilize intra-warp shuffling to efficiently construct their segment of the state table. (b) We provide a comparison of total iteration on Replica with monocular camera.

which tracks the remaining optimization iterations for each keyframe, and $\mathcal{L}_k = \{l_1, l_2, \ldots, l_k\}$ which records the last optimization loss value for each keyframe. Upon the detection of a new keyframe v_{k+1}, we update our pools as follows:

$$\mathcal{K}_{k+1} = \mathcal{K}_k \cup \{v_{k+1}\}, \tag{5}$$

$$\mathcal{R}_{k+1} = \mathcal{R}_k \cup \{r^0_{k+1}\}, \tag{6}$$

$$\mathcal{L}_{k+1} = \mathcal{L}_k \cup \{l_{k+1}\}, \tag{7}$$

where r^0_{k+1} is the initial optimization iteration count assigned to the new keyframe, and l_{k+1} is its initial optimization loss value. We then select a keyframe v' randomly from the subset of keyframes with remaining iterations, defined as $\{v_i | r_i > 0, \forall r_i \in \mathcal{R}_k\}$, to train the 3D Gaussians Map $\mathcal{G}$. Post-optimization, we decrement the optimization iteration count for the selected keyframe by one, adjusting r' to $r' - 1$, and also update the corresponding optimization loss value l'.

When $\{v_i | r_i > 0, \forall r_i \in \mathcal{R}_k\}$ is empty, we update $\mathcal{R}_k$ based on $\mathcal{L}_k$ as follows:

$$r_i = \begin{cases} 1 & l_i \notin \prod^{d_k}(\mathcal{L}_k), \\ 2 & l_i \in \prod^{d_k}(\mathcal{L}_k), \end{cases} \tag{8}$$

where $\prod^{d_k}(\cdot)$ donates top d_k largest elements, $d_k = \max(1, \frac{k}{d})$, and d is a hyperparameter. This method prioritizes keyframes with higher optimization loss values for the photorealistic rendering module, effectively tackling the long-tail optimization as demonstrated in Fig. 2.

3) Opacity Regularization: In the typical application of 3DGS, the rendered loss $\mathcal{L}_{rendered}$ is utilized to refine the 3D Gaussian primitives [8]. To efficiently manage memory usage and model size, we have devised a strategy that encourages the elimination of Gaussians in areas where they do not contribute to the rendering process. Since the presence of a Gaussian is primarily indicated by its opacity o, we impose a regularization term $\mathcal{L}_o$ on this attribute. The complete formulation of our optimization loss $\mathcal{L}$ is as follows:

$$\mathcal{L}_{rendered} = (1 - \lambda_{ssim})\mathcal{L}_1 + \lambda_{ssim}\mathcal{L}_{ssim}, \tag{9}$$

$$\mathcal{L}_o = \frac{1}{N}\sum_i |o_i|, \tag{10}$$

$$\mathcal{L} = \mathcal{L}_{rendered} + \lambda_o \mathcal{L}_o, \tag{11}$$

where λ_{ssim} is the weighting factor, λ_o is the regularization coefficient, and N denotes the total count of Gaussian primitives.

IV. Experiments

In this section, we present a comparative analysis of CaRtGS against state-of-the-art GS-SLAM systems [9], [10], [11], [16], [17] and Loopy-SLAM [28], a state-of-the-art NeRF-based SLAM system. This evaluation spans multiple scenarios, including those captured using monocular, RGB-D, and stereo cameras. Furthermore, we perform an ablation study to substantiate the efficacy of the novel techniques introduced in our approach.

A. Setup

Dataset: We conducted evaluations on three distinct camera systems: monocular, RGB-D, and stereo. These assessments were carried out on three renowned datasets: Replica [29], TUM-RGBD [30], and VECtor [31]. Replica [29] is a high-quality reconstruction dataset at room and building scale, including high-resolution high-dynamic-range (HDR) textures. TUM-RGBD [30] is a well-known RGB-D dataset that contains color and depth images captured by a Microsoft Kinect sensor, along with the ground-truth trajectory obtained from a high-accuracy motion-capture system. VECtor [31] is a SLAM benchmark dataset that covers the full spectrum of motion dynamics, environmental complexities, and illumination conditions. To ensure data consistency, we employed a soft time synchronization to align the sensor data and ground truth with a precision of $\Delta t = 0.08$ s.

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 03:37:58 UTC from IEEE Xplore. Restrictions apply.

TABLE I
QUANTITATIVE RESULTS ON REPLICA

Cam	Method	Metric	office0	office1	office2	office3	office4	room0	room1	room2
Monocular	Photo-SLAM [17]	ATE	0.20 ± 0.02	2.95 ± 6.23	0.91 ± 0.39	0.11 ± 0.01	0.17 ± 0.00	0.15 ± 0.00	0.24 ± 0.04	0.10 ± 0.02
		FPS	36.91 ± 0.75	36.41 ± 0.66	34.48 ± 0.52	34.60 ± 0.36	35.98 ± 0.49	34.40 ± 0.29	36.37 ± 0.66	33.32 ± 0.28
		IPF	2.66 ± 0.11	2.31 ± 0.05	2.30 ± 0.06	2.29 ± 0.05	2.30 ± 0.04	2.03 ± 0.02	2.22 ± 0.02	2.30 ± 0.08
		PSNR	35.02 ± 0.45	32.75 ± 5.37	31.19 ± 0.65	31.13 ± 0.53	32.94 ± 0.18	28.74 ± 0.39	30.56 ± 0.38	31.69 ± 0.25
		Points	78.40k ± 2.94k	97.04k ± 31.16k	99.40k ± 1.67k	76.36k ± 3.19k	75.98k ± 3.39k	0.11m ± 6.27k	0.12m ± 5.56k	81.10k ± 1.81k
	Ours (Photo-SLAM)	ATE	0.22 ± 0.06	2.97 ± 6.24	1.53 ± 1.37	0.12 ± 0.01	0.17 ± 0.01	0.17 ± 0.00	0.52 ± 0.48	0.09 ± 0.00
		FPS	36.65 ± 0.46	36.08 ± 0.47	33.90 ± 0.28	34.88 ± 0.68	35.96 ± 0.54	33.58 ± 0.20	36.65 ± 0.29	33.73 ± 0.26
		IPF	8.10 ± 0.21	7.76 ± 0.23	8.05 ± 0.13	7.15 ± 0.09	7.35 ± 0.13	7.40 ± 0.11	7.33 ± 0.24	7.67 ± 0.04
		PSNR	34.58 ± 0.31	34.97 ± 4.96	33.52 ± 0.12	33.26 ± 0.08	35.22 ± 0.23	31.92 ± 0.26	31.99 ± 1.15	34.39 ± 0.16
		Points	**38.32k ± 1.97k**	**48.37k ± 11.77k**	**64.07k ± 1.03k**	**54.93k ± 0.91k**	**53.67k ± 1.13k**	**87.49k ± 2.99k**	**73.44k ± 2.84k**	**58.92k ± 1.30k**
RGBD	Photo-SLAM [17]	ATE	0.45 ± 0.05	0.35 ± 0.04	1.13 ± 0.14	0.37 ± 0.02	0.44 ± 0.05	0.30 ± 0.02	0.33 ± 0.04	0.18 ± 0.00
		FPS	31.61 ± 0.53	31.96 ± 0.32	30.43 ± 0.81	29.33 ± 0.52	27.87 ± 0.54	27.49 ± 0.52	29.87 ± 0.91	27.37 ± 0.52
		IPF	3.43 ± 0.09	3.04 ± 0.12	3.18 ± 0.04	3.28 ± 0.04	3.10 ± 0.05	3.17 ± 0.05	3.12 ± 0.05	3.20 ± 0.05
		PSNR	36.83 ± 0.32	36.79 ± 0.29	32.45 ± 0.38	33.38 ± 0.07	35.13 ± 0.39	30.13 ± 2.14	33.80 ± 0.36	34.53 ± 0.87
		Points	81.34k ± 2.95k	79.24k ± 1.71k	0.12m ± 4.04k	93.03k ± 3.79k	0.12m ± 1.61k	0.19m ± 2.70k	0.16m ± 8.84k	0.14m ± 2.09k
	Ours (Photo-SLAM)	ATE	0.48 ± 0.04	0.38 ± 0.06	1.10 ± 0.19	0.38 ± 0.02	0.56 ± 0.10	0.31 ± 0.01	0.34 ± 0.03	0.18 ± 0.00
		FPS	30.84 ± 0.37	31.49 ± 0.31	30.04 ± 0.43	28.76 ± 0.58	28.64 ± 0.66	27.81 ± 0.62	29.55 ± 0.55	26.87 ± 0.31
		IPF	10.45 ± 0.30	9.90 ± 0.26	10.06 ± 0.21	10.40 ± 0.40	10.71 ± 0.35	9.95 ± 0.66	9.25 ± 0.40	9.97 ± 0.06
		PSNR	35.54 ± 0.28	37.74 ± 0.41	33.40 ± 0.29	33.84 ± 0.27	35.64 ± 0.41	29.38 ± 3.70	34.30 ± 0.64	36.54 ± 0.19
		Points	39.74k ± 1.11k	54.61k ± 2.58k	79.29k ± 3.24k	68.03k ± 2.06k	75.58k ± 4.31k	0.11m ± 3.74k	0.10m ± 1.21k	0.10m ± 2.72k
	GS-ICP-SLAM [16]	ATE	0.19 ± 0.00	0.13 ± 0.00	0.18 ± 0.00	0.19 ± 0.01	0.22 ± 0.01	0.16 ± 0.00	0.16 ± 0.00	0.11 ± 0.01
		FPS	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00
		IPF	2.88 ± 0.00	2.37 ± 0.01	2.88 ± 0.01	2.87 ± 0.00	2.91 ± 0.01	2.90 ± 0.07	2.84 ± 0.07	2.67 ± 0.01
		PSNR	40.57 ± 0.03	40.96 ± 0.11	32.77 ± 0.16	31.60 ± 0.07	38.84 ± 0.04	35.54 ± 0.06	37.81 ± 0.06	38.54 ± 0.05
		Points	1.57m ± 0.85k	1.57m ± 7.30k	1.54m ± 2.51k	1.55m ± 9.54k	1.60m ± 10.33k	1.55m ± 2.86k	1.55m ± 0.70k	1.54m ± 3.78k
	Ours (GS-ICP-SLAM)	ATE	0.25 ± 0.14	0.12 ± 0.00	0.28 ± 0.10	0.19 ± 0.02	0.24 ± 0.01	0.16 ± 0.00	0.16 ± 0.00	0.11 ± 0.00
		FPS	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00
		IPF	12.10 ± 0.07	12.36 ± 0.05	10.42 ± 0.08	10.97 ± 0.04	11.47 ± 0.05	11.03 ± 0.03	11.68 ± 0.08	10.46 ± 0.05
		PSNR	**42.60 ± 0.05**	**42.33 ± 0.03**	**36.95 ± 0.42**	**36.87 ± 0.03**	**39.77 ± 0.08**	**36.46 ± 0.10**	**39.19 ± 0.05**	**39.38 ± 0.23**
		Points	0.76m ± 7.29k	0.67m ± 5.51k	0.79m ± 17.16k	0.74m ± 12.55k	0.70m ± 16.92k	0.72m ± 16.73k	0.72m ± 16.60k	0.70m ± 13.53k

Implementation Detail: All experimental evaluations were conducted on a desktop with an *Nvidia RTX 4090 GPU*, an *AMD Ryzen 9 7950X CPU*, and *128 GB RAM*. We retained most of the original hyperparameters from the 3DGS [8]. However, we densify every 500 iterations with a positional gradients threshold $\tau_p = 0.001$ and remove the transparent Gaussians with a threshold $\epsilon_\alpha = 0.02$. By default, we set $d = 4$ and $\lambda_o = 0.001$. On Replica, we use $r^0_{k+1} = 8$, whileas $r^0_{k+1} = 2$ on TUM-RGBD and VECtor.

Evaluation: We performed all experiments 5 times to ensure statistical robustness and rendered original-resolution images for each estimated camera pose. To measure performance, we utilized the evo toolkit[1] and the torchmetrics toolkit[2]. We recorded various performance indicators, including Absolute Trajectory Error (ATE) to assess the accuracy of localization, Peak Signal-to-Noise Ratio (PSNR) to assess the quality of the photorealistic renderings, and the number of 3D Gaussian points to assess the model size. To assess the sufficiency of the Gaussian primitives' optimization, we introduced a metric known as Iterations Per Frame (IPF), defined as the ratio of total iterations to the total number of frames (IPF $= \frac{\text{Iterations}}{\text{Frames}}$). All performance indicators are reported in the format of mean $\pm$ standard deviation.

B. Results

The quantitative comparison presented in Tables I, II, and III illustrates the performance of various methods. The best resutls of the PSNR and the count of Gaussian primitives are distinctively highlighted as **1st**, 2nd, and 3rd. In summary, our approach consistently delivers superior rendering performance, utilizing a reduced number of Gaussian primitives, while adhering to real-time constraints of over 22 frames per second. Specifically, on the Replica dataset [29] with monocular camera, compared with Photo-SLAM [17], and under similar localization accuracy, our approach significantly improves the average PSNR by more than 2 dB and halves the number of Gaussian primitives. As shown in Tables I and II, our method can be easily

TABLE II
QUANTITATIVE RESULTS ON TUM-RGBD

Cam	Method	Metric	fr1/desk	fr2/xyz	fr3/office
Monocular	MonoGS [9]	ATE	4.93 ± 0.16	4.66 ± 0.13	3.35 ± 0.45
		FPS	1.87 ± 0.05	3.37 ± 0.06	2.26 ± 0.01
		IPF	84.07 ± 0.25	51.64 ± 0.26	60.5 ± 0.43
		PSNR	17.65 ± 0.40	15.56 ± 0.02	19.35 ± 0.31
		Points	**26.64k ± 1.58k**	43.59k ± 2.09k	**35.24k ± 3.24k**
	Photo-SLAM [17]	ATE	1.55 ± 0.06	0.63 ± 0.18	1.10 ± 0.70
		FPS	25.18 ± 0.30	25.83 ± 0.12	24.74 ± 0.25
		IPF	7.08 ± 0.08	6.66 ± 0.08	7.77 ± 0.20
		PSNR	19.69 ± 0.04	20.19 ± 0.52	18.32 ± 1.36
		Points	40.00k ± 0.79k	0.10m ± 7.50k	81.16k ± 3.44k
	Ours (Photo-SLAM)	ATE	1.55 ± 0.06	0.70 ± 0.08	0.57 ± 0.33
		FPS	24.95 ± 0.46	26.16 ± 0.12	25.03 ± 0.11
		IPF	17.88 ± 0.02	14.41 ± 0.26	16.06 ± 0.32
		PSNR	20.51 ± 0.08	21.54 ± 0.85	19.38 ± 1.47
		Points	38.65k ± 1.82k	66.51k ± 1.71k	51.71k ± 3.46k
RGBD	Loopy-SLAM [28]	ATE	3.93 ± 1.13	1.43 ± 0.16	4.65 ± 1.63
		FPS	0.23 ± 0.00	0.21 ± 0.00	0.20 ± 0.00
		IPF	-	-	-
		PSNR	13.66 ± 0.12	17.95 ± 0.41	17.43 ± 0.15
		Points	-	-	-
	SplaTAM [10]	ATE	2.51 ± 0.01	0.50 ± 0.00	4.52 ± 0.21
		FPS	0.27 ± 0.01	0.03 ± 0.02	0.25 ± 0.00
		IPF	460.32 ± 0.00	460.88 ± 0.00	460.84 ± 0.00
		PSNR	21.03 ± 0.10	23.19 ± 0.13	20.10 ± 0.05
		Points	0.96m ± 3.96k	6.36m ± 81.37k	0.79m ± 5.89k
	Gaussian-SLAM [11]	ATE	2.74 ± 0.11	0.96 ± 0.44	8.42 ± 1.19
		FPS	0.57 ± 0.06	0.48 ± 0.03	0.59 ± 0.02
		IPF	309.37 ± 4.29	308.44 ± 0.04	310.66 ± 0.11
		PSNR	**23.71 ± 0.10**	**23.95 ± 0.39**	**25.80 ± 0.09**
		Points	0.76m ± 12.12k	0.69m ± 26.07k	1.47m ± 6.75k
	MonoGS [9]	ATE	1.84 ± 0.09	1.71 ± 0.08	1.74 ± 0.10
		FPS	2.18 ± 0.02	3.23 ± 0.07	2.48 ± 0.03
		IPF	77.77 ± 0.06	51.23 ± 0.18	63.20 ± 0.06
		PSNR	19.00 ± 0.09	15.81 ± 0.03	19.11 ± 0.25
		Points	43.01k ± 1.95k	**37.20k ± 4.78k**	52.67k ± 2.00k
	Photo-SLAM [17]	ATE	1.49 ± 0.03	0.32 ± 0.02	1.17 ± 0.34
		FPS	23.45 ± 0.18	23.44 ± 0.01	22.63 ± 0.22
		IPF	8.88 ± 0.14	7.68 ± 0.28	8.54 ± 0.26
		PSNR	19.98 ± 0.03	21.92 ± 0.42	22.18 ± 1.20
		Points	45.64k ± 1.18k	68.68k ± 10.00k	67.69k ± 1.75k
	Ours (Photo-SLAM)	ATE	1.52 ± 0.03	0.30 ± 0.01	0.90 ± 0.03
		FPS	23.06 ± 0.22	23.36 ± 0.07	22.78 ± 0.10
		IPF	20.60 ± 0.46	18.05 ± 0.31	17.66 ± 0.32
		PSNR	20.54 ± 0.06	22.75 ± 0.22	22.95 ± 0.79
		Points	38.65k ± 0.76k	49.80k ± 2.63k	71.33k ± 6.79k
	GS-ICP-SLAM [16]	ATE	3.26 ± 0.28	2.26 ± 0.04	3.07 ± 0.41
		FPS	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00
		IPF	6.10 ± 0.05	3.69 ± 0.05	3.96 ± 0.08
		PSNR	15.62 ± 0.07	18.43 ± 0.19	19.20 ± 0.05
		Points	0.53m ± 6.82k	1.91m ± 11.37k	2.09m ± 21.04k
	Ours (GS-ICP-SLAM)	ATE	3.92 ± 0.71	2.44 ± 0.06	4.11 ± 1.28
		FPS	30.00 ± 0.00	30.00 ± 0.00	30.00 ± 0.00
		IPF	20.02 ± 0.10	18.43 ± 0.15	12.17 ± 0.13
		PSNR	17.54 ± 0.07	21.35 ± 0.20	20.84 ± 0.06
		Points	0.18m ± 3.65k	0.13m ± 12.32k	0.34m ± 19.24k

[1]https://github.com/MichaelGrupp/evo
[2]https://github.com/Lightning-AI/torchmetrics

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 03:37:58 UTC from IEEE Xplore. Restrictions apply.

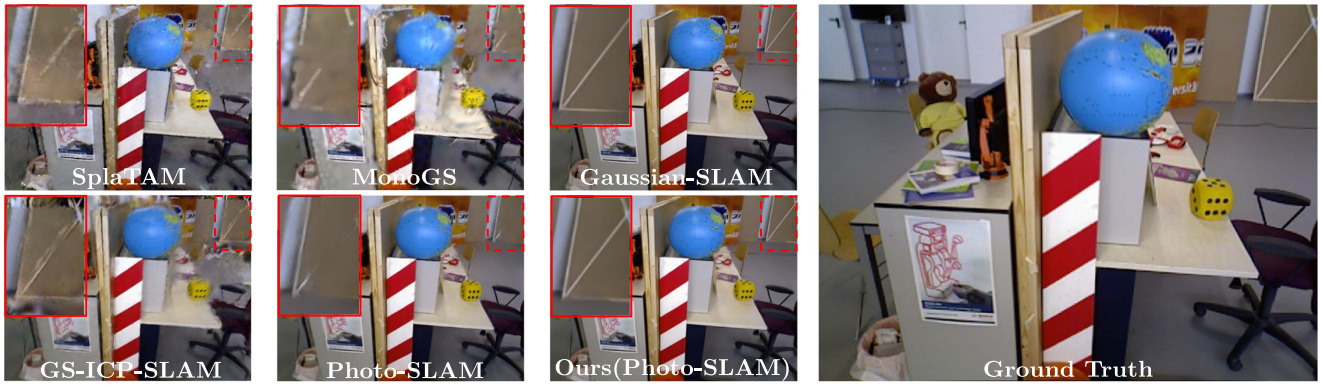


Fig. 5. **Qualitative results on TUM-RGBD with RGBD camera.** Qualitative assessments demonstrate that our approach significantly improves rendering quality and effectively mitigates visual artifacts. Furthermore, our method achieves precise localization accuracy. In contrast, Gaussian-SLAM exhibits substantial drift, as indicated by the red dashed line.

TABLE III
QUANTITATIVE RESULTS ON VECTOR

Cam	Method	Metric	corner-slow	robot-normal	corridors-dolly
Monocular	Photo-SLAM [17]	ATE	0.66 ± 0.01	2.20 ± 1.66	9.56 ± 6.08
		FPS	23.27 ± 0.21	21.90 ± 0.32	20.18 ± 0.26
		IPF	3.11 ± 0.03	3.37 ± 0.17	3.11 ± 0.03
		PSNR	24.63 ± 0.05	19.58 ± 0.18	15.31 ± 0.69
		Points	0.12m ± 17.02k	0.16m ± 72.38k	0.38m ± 3.99k
	Ours (Photo-SLAM)	ATE	0.68 ± 0.02	2.35 ± 1.17	10.06 ± 6.20
		FPS	21.56 ± 0.33	18.30 ± 1.20	18.00 ± 0.26
		IPF	7.69 ± 0.17	10.78 ± 0.68	11.11 ± 0.18
		PSNR	**25.37 ± 0.12**	**22.16 ± 1.46**	**23.02 ± 5.67**
		Points	7.31k ± 0.25k	8.24k ± 2.06k	36.96k ± 1.59k
Stereo	Photo-SLAM [17]	ATE	1.15 ± 0.00	1.52 ± 0.00	11.91 ± 0.04
		FPS	20.43 ± 0.32	17.77 ± 0.31	19.31 ± 0.01
		IPF	1.68 ± 0.08	2.58 ± 0.04	2.76 ± 0.02
		PSNR	19.34 ± 0.02	16.59 ± 0.01	14.51 ± 0.34
		Points	38.98k ± 4.29k	47.36k ± 0.64k	0.24m ± 2.92k
	Ours (Photo-SLAM)	ATE	1.15 ± 0.00	1.52 ± 0.00	11.51 ± 0.23
		FPS	20.75 ± 0.37	14.64 ± 0.23	16.64 ± 0.83
		IPF	9.23 ± 0.02	12.24 ± 0.20	11.21 ± 0.16
		PSNR	19.56 ± 0.04	16.77 ± 0.05	19.34 ± 0.06
		Points	**6.45k ± 0.20k**	**7.68k ± 0.24k**	**30.81k ± 2.21k**

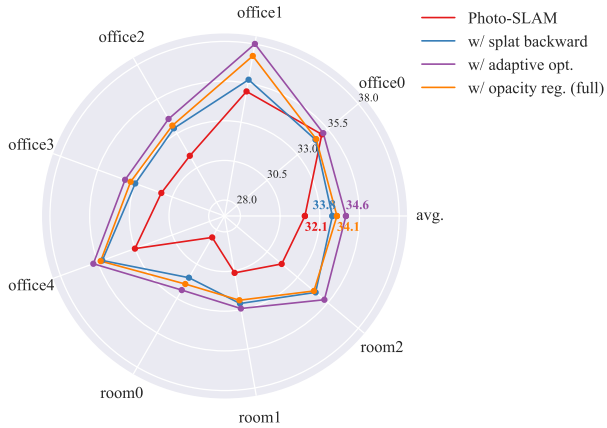


Fig. 6. **The radar chart of ablation study.** Radial axis presents the PSNR.

integrated into Photo-SLAM [17] and GS-ICP-SLAM [16]. In Table II, our approach achieves high rendering quality using a comparable number of Gaussian primitives to MonoGS [9]. In Table III, we present the results on VECtor [31], specifically using a monocular camera. Our method improves the average PSNR by more than 3 dB with only one-tenth of the Gaussian primitives. Furthermore, the qualitative results depicted in Fig. 5 corroborate that our approach achieves high-fidelity rendering.

Fig. 6 depicts our ablation studies on the monocular Replica dataset [29], rigorously validating our design choices and highlighting their contributions to system performance. Key findings include:

Splat-wise backpropagation enhances the rendering quality by refining the iterative process efficiently: The integration of splat-wise backpropagation has significantly improved average total iterations from 4.6 k to 15.4 k and average PSNR from 32.1 dB to 33.8 dB.

Adaptive optimization strategically allocates computational resources to enhance rendering quality: Integrating splat-wise backpropagation with adaptive optimization has continuously boosted average PSNR from 33.8 dB to 34.6 dB. Furthermore, as illustrated in Fig. 2, this approach equitably distributes computational resources across keyframes, efficiently addressing long-tail optimization challenges.

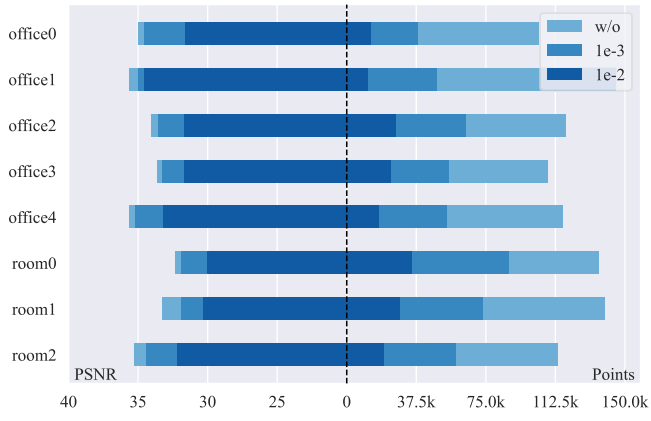


Fig. 7. **The effect of opacity regularization.** The left side illustrates the value of PSNR. The right side depicts the count of Gaussian points.

Opacity regularization is instrumental in reducing the model size without compromising the superior rendering quality: Our opacity regularization technique, as shown in Fig. 7, can halve the model size with a regularization coefficient of $\lambda_o = 0.001$, with minimal PSNR performance loss. Increasing the coefficient to 0.01 further reduces less critical Gaussian primitives, which

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 03:37:58 UTC from IEEE Xplore. Restrictions apply.

results in a more efficient model at the expense of some rendering quality.

V. Limitations and Feature Work

CaRtGS is an adaptive optimization technology that leverages 3D Gaussian models for high-quality rendering and environmental reconstruction in real-time GS-SLAM systems. Despite its potential, several limitations and challenges are identified below, structured into categories for clarity:

1) *Dynamic Environment Challenges:* CaRtGS assumes static environments, limiting real-world use and causing tracking failures with dynamic objects.
2) *Localization Robustness:* CaRtGS focuses on improving the rendering quality of GS-SLAM. However, localization accuracy affects rendering quality, especially in some degeneracy scenarios. Therefore, a robustness localization module is essential for GS-SLAM.
3) *Geometry Accuracy:* Effective geometry mapping is vital in GS-SLAM. As shown in Table III, the stereo model's inferior rendering quality stems from the stereo camera's suboptimal geometry mapping.

Looking forward, we envision further improvements by integrating advanced machine learning models to predict and handle dynamic objects.

VI. Conclusion

In this work, we introduced CaRtGS, a novel framework that integrates computational alignment with Gaussian Splatting SLAM to achieve real-time photorealistic dense rendering. Our key contribution lies in the development of an adaptive computational alignment strategy that optimizes the rendering process by addressing the computational misalignment inherent in GS-SLAM systems. Through fast splat-wise backpropagation, adaptive optimization, and opacity regularization, we significantly enhanced the rendering quality and computational efficiency of the SLAM process.

References

[1] Z. Teed and J. Deng, "DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 16558–16569.

[2] A. Segal, D. Haehnel, and S. Thrun, "Generalized-ICP," in *Proc. Robot.: Sci. Syst.*, Seattle, WA, USA, 2009, vol. 2, no. 4, Paper 435.

[3] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual–inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.

[4] S. Zhong et al., "CoLRIO: LiDAR-ranging-inertial centralized state estimation for robotic swarms," in *Proc. IEEE Int. Conf. Robot. Automat.*. IEEE, 2024, pp. 3920–3926.

[5] D. Feng et al., "S3E: A multi-robot multimodal dataset for collaborative SLAM," *IEEE Robot. Automat. Lett.*, vol. 9, no. 12, pp. 11401–11408, Dec. 2024.

[6] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, pp. 99–106, 2021.

[7] F. Tosi et al., "How NeRFs and 3D Gaussian splatting are reshaping SLAM: A survey," 2024, arXiv: *2402.13255*.

[8] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–14, 2023.

[9] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison, "Gaussian splatting SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 18039–18048.

[10] N. Keetha et al., "SplaTAM: Splat track & map 3D Gaussians for dense RGB-D SLAM," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 21357–21366.

[11] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, "Gaussian-SLAM: Photo-realistic dense slam with Gaussian splatting," 2023, arXiv: *2312.10070*.

[12] L. Zhu, Y. Li, E. Sandström, K. Schindler, and I. Armeni, "LoopSplat: Loop closure by registering 3D Gaussian splats," 2024, arXiv: *2408.10154*.

[13] E. Sandström et al., "Splat-SLAM: Globally optimized RGB-only SLAM with 3D Gaussians," 2024, arXiv: *2405.16544*.

[14] F. A. Sarikamis and A. A. Alatan, "IG-SLAM: Instant Gaussian SLAM," 2024, arXiv: *2408.01126*.

[15] Z. Peng et al., "RTG-SLAM: Real-time 3D reconstruction at scale using Gaussian splatting," in *Proc. ACM SIGGRAPH 2024 Conf. Papers*, 2024, pp. 1–11.

[16] S. Ha, J. Yeon, and H. Yu, "RGBD GS-ICP SLAM," in *Proc. Eur. Conf. Comput. Vis.*. Springer, 2024, pp. 180–197.

[17] H. Huang, L. Li, H. Cheng, and S.-K. Yeung, "Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular stereo and RGB-D cameras," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 21584–21593.

[18] S. Kheradmand et al., "3D Gaussian splatting as Markov chain Monte Carlo," 2024, arXiv: *2404.09591*.

[19] T. Lu et al., "Scaffold-GS: Structured 3D Gaussians for view-adaptive rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 20654–20664.

[20] S. S. Mallick et al., "Taming 3DGS: High-quality radiance fields with limited resources," in *Proc. SIGGRAPH Asia 2024 Conf. Papers*, 2024, pp. 1–11.

[21] S. Durvasula, A. Zhao, F. Chen, R. Liang, P. K. Sanjaya, and N. Vijaykumar, "DISTWAR: Fast differentiable rendering on raster-based rendering pipelines," 2023, arXiv: *2401.05345*.

[22] L. Höllein, A. Božič, M. Zollhöfer, and M. Nießner, "3DGS-LM: Faster Gaussian-splatting optimization with Levenberg-Marquardt," 2024, arXiv: *2409.12892*.

[23] G. Feng et al., "FlashGS: Efficient 3D Gaussian splatting for large-scale and high-resolution rendering," 2024, arXiv: *2408.07967*.

[24] Z. Fan et al., "InstantSplat: Unbounded sparse-view pose-free Gaussian splatting in 40 seconds," 2024, arXiv: *2403.20309*.

[25] S. Niedermayr, J. Stumpfegger, and R. Westermann, "Compressed 3D Gaussian splatting for accelerated novel view synthesis," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 10349–10358.

[26] W. Morgenstern, F. Barthel, A. Hilsmann, and P. Eisert, "Compact 3D scene representation via self-organizing Gaussian grids," in *Proc. Eur. Conf. Comput. Vis.*. Springer, 2024, pp. 18–34.

[27] H. Wang et al., "End-to-end rate-distortion optimized 3D Gaussian representation," in *Proc. Eur. Conf. Comput. Vis.*. Springer, 2024, pp. 76–92.

[28] L. Liso, E. Sandström, V. Yugay, L. Van Gool, and M. R. Oswald, "Loopy-SLAM: Dense neural SLAM with loop closures," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 20363–20373.

[29] J. Straub et al., "The Replica dataset: A digital replica of indoor spaces," 2019, arXiv: *1906.05797*.

[30] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of RGB-D SLAM systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*. IEEE, 2012, pp. 573–580.

[31] L. Gao et al., "VECtor: A versatile event-centric benchmark for multi-sensor SLAM," *IEEE Robot. Automat. Lett.*, vol. 7, no. 3, pp. 8217–8224, Jul. 2022.

Authorized licensed use limited to: SUN YAT-SEN UNIVERSITY. Downloaded on December 22,2025 at 03:37:58 UTC from IEEE Xplore. Restrictions apply.